

Lab 10: Explainability with Alibi

In this lab, you'll learn about the Alibi Explain library and implement global and local explanations of tabular and image classification models.

Alibi Explain is an open source Python library aimed at machine learning model inspection and interpretation. The focus of the library is to provide high-quality implementations of black-box, white-box, local and global explanation methods for classification and regression models.

Complete all the Deliverables mentioned below and show it to a TA for credit.

Deliverables

- Finish all the TODOs in Section 2
- Generate PD Plots. Discuss your findings about model performance with the TA and answer Q1, Q2, and Q3:
 - Q1: Which features have higher importance in the prediction? (Section 2.1)
 - Q2: What can you conclude with the PD plots? (Section 2.2)
 - Q3: Discuss with the TA the results (heatmaps). (Section 2.2)
- Show final results about the Anchor parameters and discuss them with the TA.

Getting Started

Clone this repository and run all cells in the notebook.

Install Dependencies

For this assignment, make sure you have the required packages installed.

```
pip install -r requirements.txt
```

(If there are any major unsolvable issues prefer running this notebook on Google Colaboratory)

Possible Issues with installing Alibi

`TypeError: issubclass() arg 1 must be a class`

Solution: <https://stackoverflow.com/questions/76313592/import-langchain-error-typeerror-issubclass-arg-1-must-be-a-class>

If there's any more issues, please contact a TA to update this list (with a solution if its solved)

```
In [73]: import numpy as np
import pandas as pd

from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

from alibi.explainers import PartialDependence, plot_pd, plot_pd_variance
from alibi.explainers import PartialDependenceVariance
```

Section 1 - Data + Modeling

In this section, we will be setting up our data, perform some preprocessing and train a model.

1.1 Fetching the Dataset

We are going to use the The Boston Housing Dataset in the next experiment.

<https://www.cs.toronto.edu/~delve/data/boston/bostonDetail.html>

There are 14 attributes in each case of the dataset:

- CRIM - per capita crime rate by town
- ZN - proportion of residential land zoned for lots over 25,000 sq.ft.
- INDUS - proportion of non-retail business acres per town.
- CHAS - Charles River dummy variable (1 if tract bounds river; 0 otherwise)

- NOX - nitric oxides concentration (parts per 10 million)
- RM - average number of rooms per dwelling
- AGE - proportion of owner-occupied units built prior to 1940
- DIS - weighted distances to five Boston employment centres
- RAD - index of accessibility to radial highways
- TAX - full-value property-tax rate per \$10,000
- PTRATIO - pupil-teacher ratio by town
- B - $1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town
- LSTAT - % lower status of the population
- MEDV - Median value of owner-occupied homes in \$1000's

```
In [74]: df = pd.read_csv('BostonHousing.csv')
df.head()
```

```
Out[74]:
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

1.2 Dataset Preprocessing

Run the following code necessary to pre-process the data for alibi.

```
In [75]: # extract feature names
feature_names = df.columns.tolist()
target_feature = 'medv'
feature_names.remove(target_feature)

# define target names
target_names = [target_feature]
```

```

# define categorical columns
categorical_columns_names = []

# define categorical and numerical indices for later preprocessing
categorical_columns_indices = [feature_names.index(cn) for cn in categorical_columns_names]
numerical_columns_indices = [feature_names.index(fn) for fn in feature_names if fn not in categorical_columns_names]

# extract data
X = df[feature_names]
# define target column
y = df[target_feature]

# split data in train & test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

```

```

In [76]: # define and fit the ordinal encoder
oe = OrdinalEncoder().fit(X_train[categorical_columns_names])

# transform the categorical columns to ordinal encoding
X_train.loc[:, categorical_columns_names] = oe.transform(X_train[categorical_columns_names])
X_test.loc[:, categorical_columns_names] = oe.transform(X_test[categorical_columns_names])

# convert data to numpy
X_train, y_train = X_train.to_numpy(), y_train.to_numpy()
X_test, y_test = X_test.to_numpy(), y_test.to_numpy()

# define categorical mappings
categorical_names = {i: list(v) for (i, v) in zip(categorical_columns_indices, oe.categories_)}

```

```

In [77]: # define numerical standard scaler
num_transf = StandardScaler()

# define categorical one-hot encoder
cat_transf = OneHotEncoder(
    categories=[range(len(x)) for x in categorical_names.values()],
    handle_unknown='ignore',
)

# define preprocessor
preprocessor = ColumnTransformer(

```

```
transformers=[
    ('cat', cat_transf, categorical_columns_indices),
    ('num', num_transf, numerical_columns_indices),
],
sparse_threshold=0
)
```

```
In [78]: # fit preprocessor
preprocessor.fit(X_train)

# preprocess train and test datasets
X_train_ohe = preprocessor.transform(X_train)
X_test_ohe = preprocessor.transform(X_test)
```

1.3 Training the Model

Run the cell to fit the model to the dataset

```
In [79]: # define and fit regressor - feel free to play with the hyperparameters
predictor = RandomForestRegressor(random_state=0)
predictor.fit(X_train_ohe, y_train)

# compute scores
print('Train score: %.2f' % (predictor.score(X_train_ohe, y_train)))
print('Test score: %.2f' % (predictor.score(X_test_ohe, y_test)))
```

Train score: 0.98

Test score: 0.77

Section 2 - Explainability with Alibi

In this section, we will be finally using Alibi to explain our trained model using different techniques. Fill in the TODOs and generate plots where required.

```
In [80]: # Creating a Prediction Function - Includes pipeline from preprocessing to prediction
prediction_fn = lambda x: predictor.predict(preprocessor.transform(x))
```

Computing Explanations

We will finally be computing explanations for selected features. Play around with different methods.

2.1 Partial Dependence Variance (PDV) - Feature Importance

Partial Dependence Variance is a method to compute the global feature importance or the feature interaction of a pair of features.

Complete the code below and answer Q1

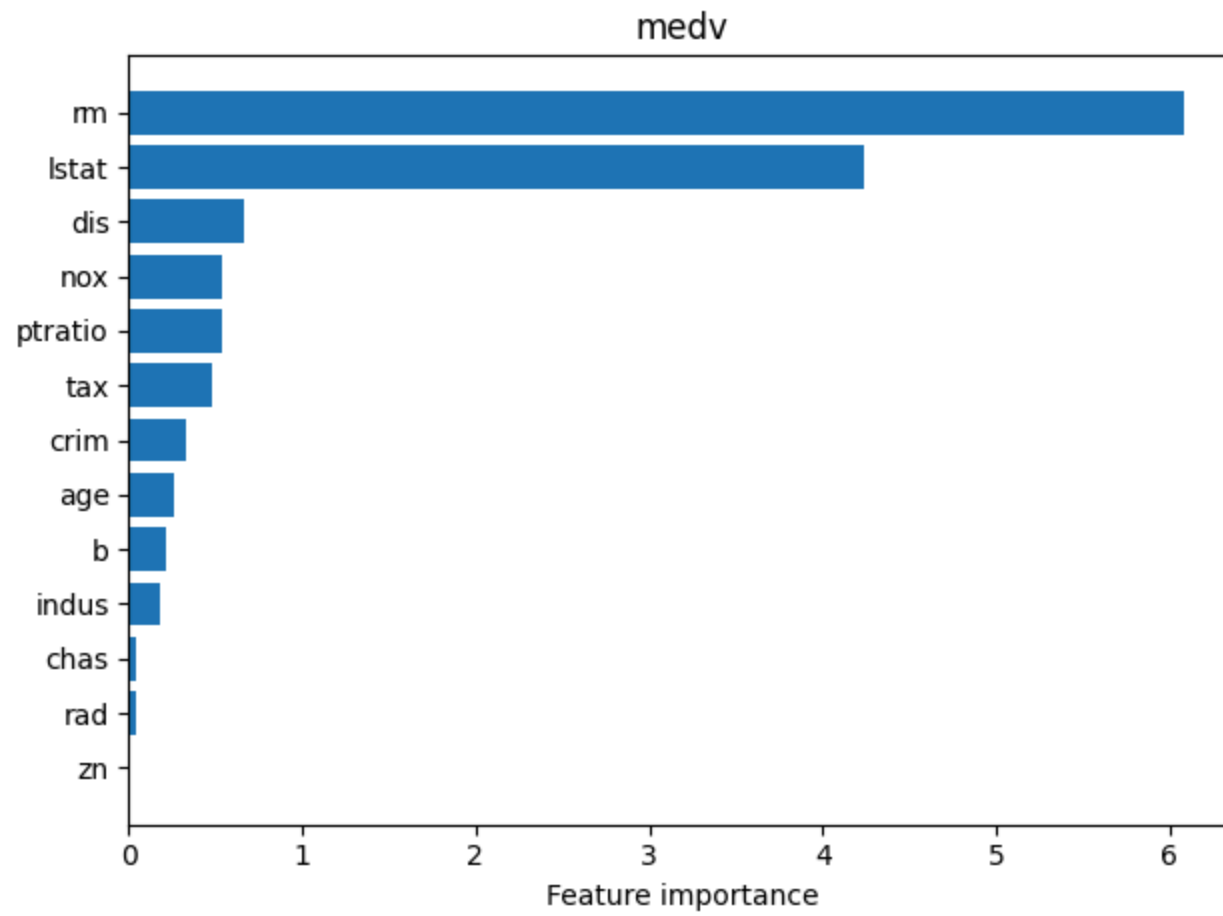
Q1: Which features have higher importance in the prediction?

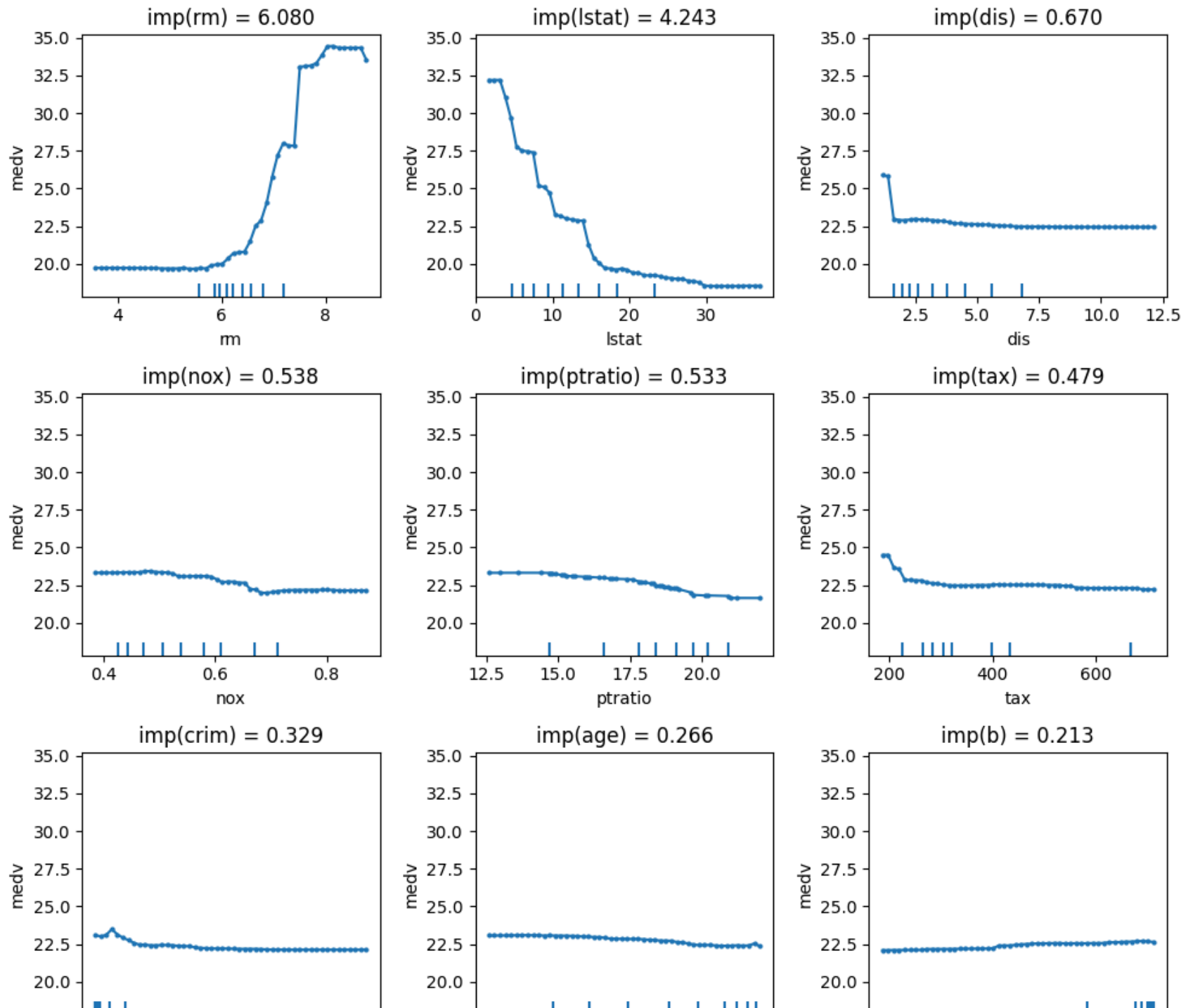
```
In [81]: # TODO: Determine which features are more important
# suggestion: Use PartialDependenceVariance class and explain function using the argument method='importance'

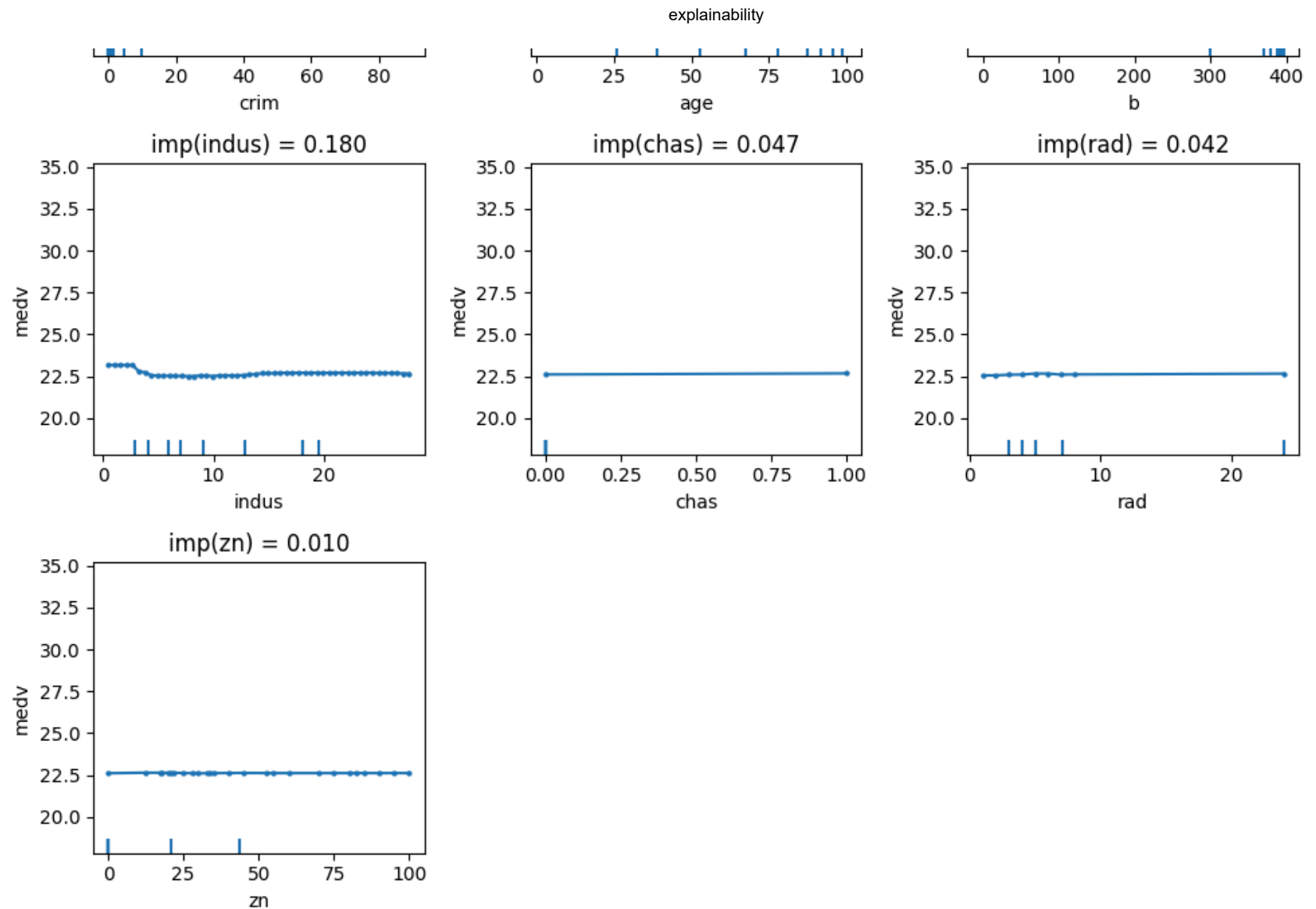
explainer = PartialDependenceVariance(predictor=prediction_fn, feature_names=feature_names, categorical_names=categorical_names)
exp_importance = explainer.explain(X=X_train, method='importance', grid_resolution=50)

# plot the results using
plot_pd_variance(exp=exp_importance)
plot_pd_variance(exp=exp_importance, summarise=False, fig_kw={'figheight': 15, 'figwidth': 10})
```

```
Out[81]: array([[<Axes: title={'center': 'imp(rm) = 6.080'}, xlabel='rm', ylabel='medv'>,
  <Axes: title={'center': 'imp(lstat) = 4.243'}, xlabel='lstat', ylabel='medv'>,
  <Axes: title={'center': 'imp(dis) = 0.670'}, xlabel='dis', ylabel='medv'>],
  [<Axes: title={'center': 'imp(nox) = 0.538'}, xlabel='nox', ylabel='medv'>,
  <Axes: title={'center': 'imp(ptratio) = 0.533'}, xlabel='ptratio', ylabel='medv'>,
  <Axes: title={'center': 'imp(tax) = 0.479'}, xlabel='tax', ylabel='medv'>],
  [<Axes: title={'center': 'imp(crim) = 0.329'}, xlabel='crim', ylabel='medv'>,
  <Axes: title={'center': 'imp(age) = 0.266'}, xlabel='age', ylabel='medv'>,
  <Axes: title={'center': 'imp(b) = 0.213'}, xlabel='b', ylabel='medv'>],
  [<Axes: title={'center': 'imp(indus) = 0.180'}, xlabel='indus', ylabel='medv'>,
  <Axes: title={'center': 'imp(chas) = 0.047'}, xlabel='chas', ylabel='medv'>,
  <Axes: title={'center': 'imp(rad) = 0.042'}, xlabel='rad', ylabel='medv'>],
  [<Axes: title={'center': 'imp(zn) = 0.010'}, xlabel='zn', ylabel='medv'>,
  None, None]], dtype=object)
```







A1: Partial Dependence Variance (PDV) measures the extent to which the model's predicted result shifts as individual features change, with all other features maintained at their average levels. In other terms, it assesses the sensitivity of the model's predictions to variations in a specific feature, not just the correlations. Features that have higher PDV values have a greater overall impact on the model's output. Based on the PDV plots I have created for the Boston Housing dataset, the features that demonstrate the greatest significance are:

- **rm** (average number of rooms per dwelling): This characteristic has the most substantial beneficial impact on the model's forecasts. With an increase in the average room count, the 'medv' (median value of owner-occupied homes) significantly escalates. This is logically understandable, since homes with additional rooms are generally bigger and worth more.
- **lstat** (% lower status of the population): This characteristic shows a significant inverse correlation with property values. Regions featuring a greater proportion of lower-status inhabitants usually exhibit considerably reduced property values. The predictions made by the model are very responsive to this feature, resulting in a high PDV score.
- **dis** (weighted distance to five employment centers in Boston): This variable reflects the appeal of location and ease of access. The PDV findings indicate that moderate proximity to job centers correlates with increased home prices, whereas extremely short or excessively long distances lower those values. This demonstrates the equilibrium between closeness to employment and the attractiveness of more tranquil suburban regions.
- **nox** (nitric oxides concentration in parts per 10 million): This attribute is highly significant, demonstrating an inverse relationship with 'medv'. Higher pollution levels are linked to less desirable neighborhoods, leading to a decline in property values.
- **ptratio** (pupil-teacher ratio by town): This attribute represents educational quality through the ratio of teachers to pupils. A greater ratio, meaning one teacher supervising fewer students, is viewed as an indicator of school quality because it allows the teacher to focus more on each child, which influences the local housing market, as families often choose to reside near their children's schools.

2.2 Partial Dependence (PD)

Partial Dependence is a method to visualize the marginal effect that one or two features have on the predicted outcome of a machine learning model.

By inspecting the PD plots, one can understand whether the relation between a feature/pair of features is, for example, a simple linear or quadratic relation, whether it presents a monotonically increasing or decreasing trend, or reveal a more complex response.

Computing Explanations

We will finally be computing explanations. You can try different modes of `kind` to see different visualizations. (Suggestion: use `both` to visualize all data)

As per the documentation,

`kind` - If set to `average`, then only the partial dependence (PD) averaged across all samples from the dataset is returned. If set to `individual`, then only the individual conditional expectation (ICE) is returned for each individual from the dataset. Otherwise, if set to `both`, then both the PD and the ICE are returned.

Run the following code with all features and also selecting some specific features.

Q2: What can you conclude?

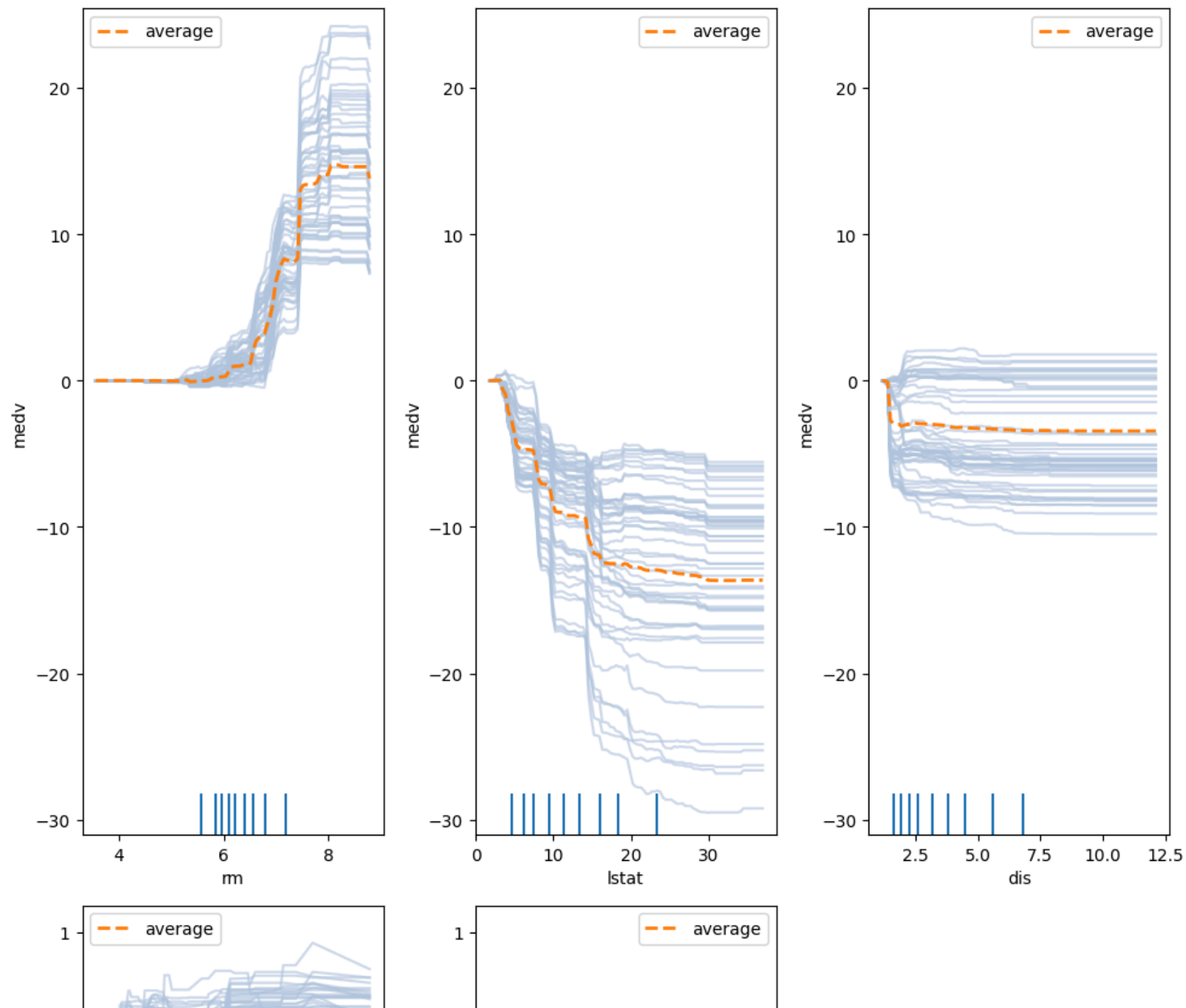
```
In [82]: # Define explainer
explainer_pd = PartialDependence(predictor=prediction_fn,
                                feature_names=feature_names,
                                categorical_names=categorical_names,
                                target_names=target_names)
```

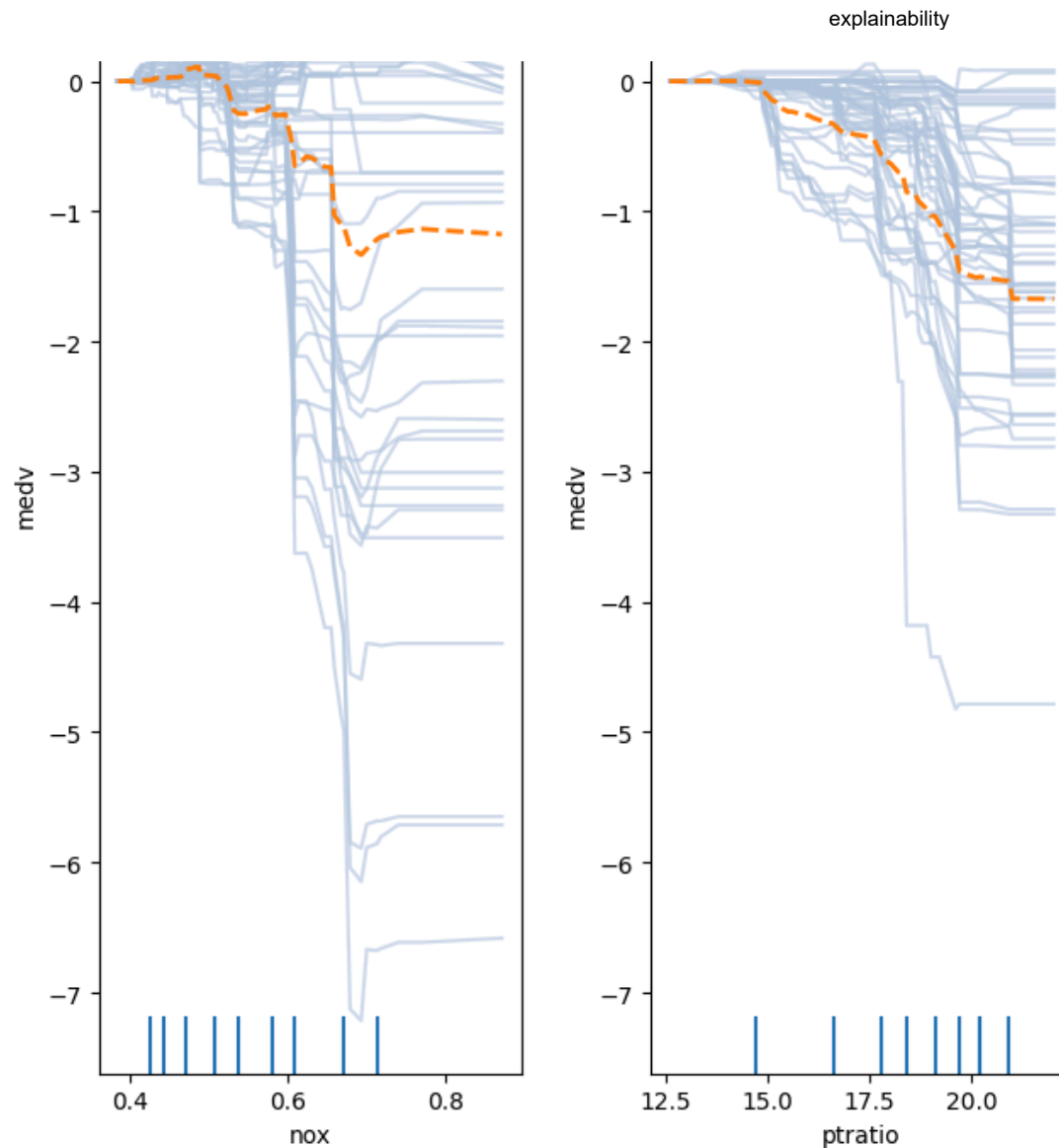
```
In [83]: # you can also compute explanations for selected features.
# Hint - Select Column Indices = feature_names.index("COLUMN_NAME")
# zn    indus  chas  nox    rm    age    dis    rad    tax    ptratio b    lstat  medv

# TODO - Select features you wish to compute explanations for
# Hint - Select Column Indices = feature_names.index("COLUMN_NAME")
features = [feature_names.index("rm"), feature_names.index("lstat"), feature_names.index("dis"), feature_names.index("dis")]
```

```
In [84]: # compute explanations
exp = explainer_pd.explain(X=X_train,
                          features=features,
                          kind='both') # kind = [both, individual, average]

# plot partial dependence curves
# this setup should work for any mode of kind, feel free to alter this to better suit your plot view.
plot_pd(exp=exp,
        n_cols=3,
        n_ice=50,
        sharey='row',
        center = True,
        fig_kw={'figheight': 15, 'figwidth': 10});
```





A2: The Partial Dependence (PD) plots show how the model's predicted median value of owner-occupied homes in \$1000s (MEDV) changes as individual features vary, while all other features are averaged out. From these plots, we can observe the shape and strength of the relationships between key features and the target variable:

- rm (average number of rooms per dwelling): The relationship is monotonically increasing and roughly linear up to around 5 rooms, then begins to flatten slightly for higher values (around 7 rooms). This indicates that adding more rooms generally

increases house value, but the marginal benefit diminishes for very large homes, introducing a mild saturation effect.

- lstat (% lower status of the population): This feature exhibits a strong negative, nonlinear relationship with 'medv'. At lower 'lstat' values, the impact on price is modest, but beyond roughly 15%, the home values drop sharply and remain low. This suggests that once socioeconomic disadvantage crosses a threshold, predicted home values decline drastically.
- dis (weighted distance to five employment centers in Boston): The relationship is initially negative, then stabilizes at lower price levels. As 'dis' increases, indicating greater distance from employment centers, 'medv' tends to decrease, showing that homes located farther from economic hubs are generally less valuable due to reduced accessibility.
- nox (nitric oxides concentration in parts per 10 million): Exhibits a steadily declining, approximately exponential pattern. As pollution increases, residential property values initially drop significantly and then stabilize, suggesting that the model reflects a considerable environmental cost for elevated NOX levels.
- ptratio (pupil-teacher ratio by town): Exhibits a slight negative linear correlation. Increased student-teacher ratios (indicating lower school quality) are linked to marginally lower expected home values.

These patterns align with actual housing dynamics, where structural quality (e.g., RM), social status (LSTAT), environmental factors (NOX), and accessibility (DIS) interact in intricate yet understandable ways to influence property values.

Check Partial Dependence for Two Features

Choose pairs of features to visualize their relationships and interactions with each other. These plots may be a bit confusing so try your best to explain whatever you understand to the TA. We suggest to select a feature with high and low importance. You can also try different combinations of features in order to get better conclusion.

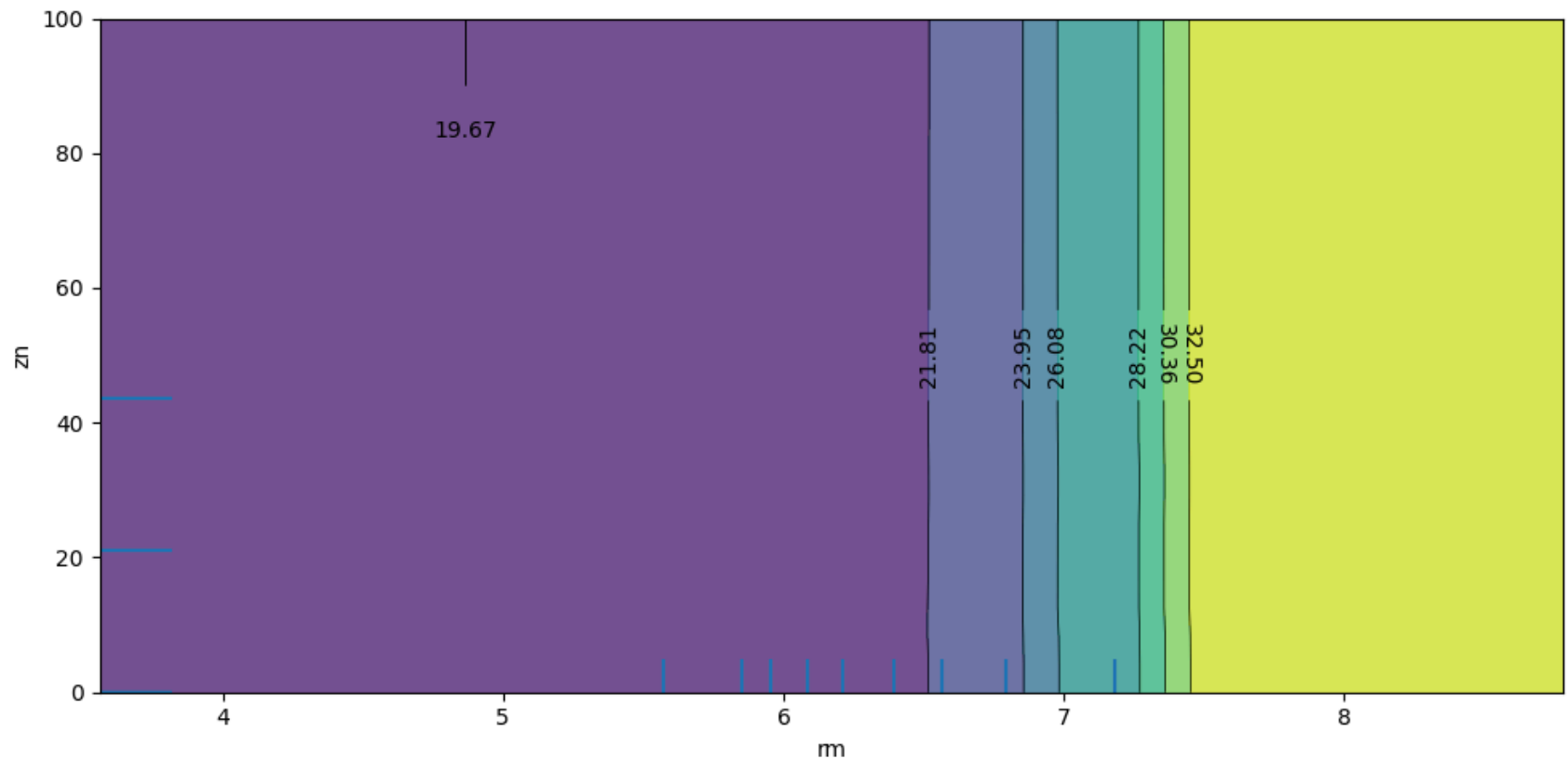
Q3: Discuss with the TA the results.

```
In [85]: # TODO: select combination of features
FEATURE1 = 'rm'
FEATURE2 = 'zn'
```

```
In [86]: combined_features = [(feature_names.index(FEATURE1), feature_names.index(FEATURE2))]

# compute explanations
comb_exp = explainer_pd.explain(X=X_train,
                               features=combined_features,
                               kind='average',
                               grid_resolution=25) # kind = [both, individual, average]
```

```
# plot partial dependence curves
plot_pd(exp=comb_exp,
        n_cols=3,
        n_ice=50,
        sharey='row',
        center = True,
        fig_kw={'figheight': 5, 'figwidth': 10});
```



A3: The 2D Partial Dependence plot for 'rm' and 'zn' shows how their combined effect influences the predicted median home value (MEDV). The color gradient indicates that as 'rm' increases, predicted home values rise sharply, showing a strong positive relationship. In contrast, 'zn' has very little impact on the model's predictions judging by how the color changes occur mostly horizontally along the 'rm' axis, not vertically along 'zn'. This suggests that the average number of rooms per dwelling (rm) is a far more influential feature than proportion of residential land zoned for lots over 25,000 sq.ft. (zn). The nearly vertical contour lines confirm that 'rm' drives most of the variation, while 'zn' remains relatively flat across its range. Hence, house size found through 'rm'

dominates the prediction of home values, while 'zn' has a weak or negligible influence. The model predicts higher prices for homes with more rooms, largely independent of the proportion of residential land zoned.

In the TODO below, I have tried out quickshift and felzenszwalb segmentation functions along with the slic segmentation function. Since they require different arguments, I changed the kwargs. Feel free to uncomment them and run them to check the results on those functions. Overall, I have found slic segmentation function with 15 or 5 segments to be the most satisfying results (personally speaking).

2.3 Anchors

This algorithm provides model-agnostic (black box) and human interpretable explanations suitable for classification models applied to images, text and tabular data. The idea behind anchors is to explain the behaviour of complex models with high-precision rules called anchors. These anchors are locally sufficient conditions to ensure a certain prediction with a high degree of confidence. Run all the cells and try different parameters to see how this algorithm works.

We will be explaining predictions from the ImageNet model on the cats dataset.

```
In [87]: import tensorflow as tf
import matplotlib

import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras.applications.inception_v3 import InceptionV3, preprocess_input, decode_predictions
from alibi.datasets import load_cats
from alibi.explainers import AnchorImage
```

```
In [88]: model = InceptionV3(weights='imagenet')
```

```
In [89]: image_shape = (299, 299, 3)
data, labels = load_cats(target_size=image_shape[:2], return_X_y=True)
print(f'Images shape: {data.shape}')
```

Images shape: (4, 299, 299, 3)

```
In [90]: images = preprocess_input(data)
preds = model.predict(images)
```



```
label = decode_predictions(preds, top=3)
print(label[0])
```

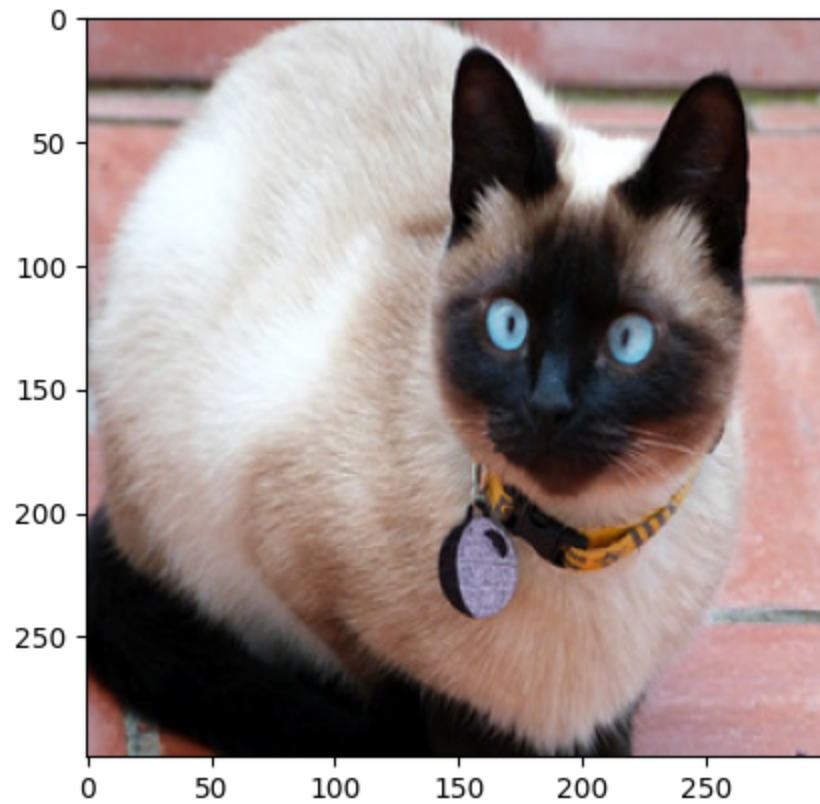
```
1/1 [=====] - 1s 728ms/step
[('n02123045', 'tabby', 0.82086843), ('n02123159', 'tiger_cat', 0.14372909), ('n02124075', 'Egyptian_cat', 0.01642176)]
1/1 [=====] - 1s 728ms/step
[('n02123045', 'tabby', 0.82086843), ('n02123159', 'tiger_cat', 0.14372909), ('n02124075', 'Egyptian_cat', 0.01642176)]
```

```
In [91]: # Define prediction function
predict_fn = lambda x: model.predict(x)
```

```
In [92]: #TODO:play around with parameters
segmentation_fn = 'slic' # choose from different segmentation functions - https://scikit-image.org/docs/dev/api/skim
# segmentation_fn = 'quickshift'
# segmentation_fn = 'felzenszwalb'
kwargs = {'n_segments': 15, 'compactness': 20, 'sigma': .5, 'start_label': 0} # slic
# kwargs = {'ratio': 1.0, 'kernel_size': 5, 'max_dist': 10, 'sigma': 0} # quickshift
# kwargs = {'scale': 1, 'sigma': .5} # felzenszwalb
explainer = AnchorImage(predict_fn, image_shape, segmentation_fn=segmentation_fn,
                        segmentation_kwargs=kwargs, images_background=None)
```

```
1/1 [=====] - 0s 101ms/step
1/1 [=====] - 0s 101ms/step
```

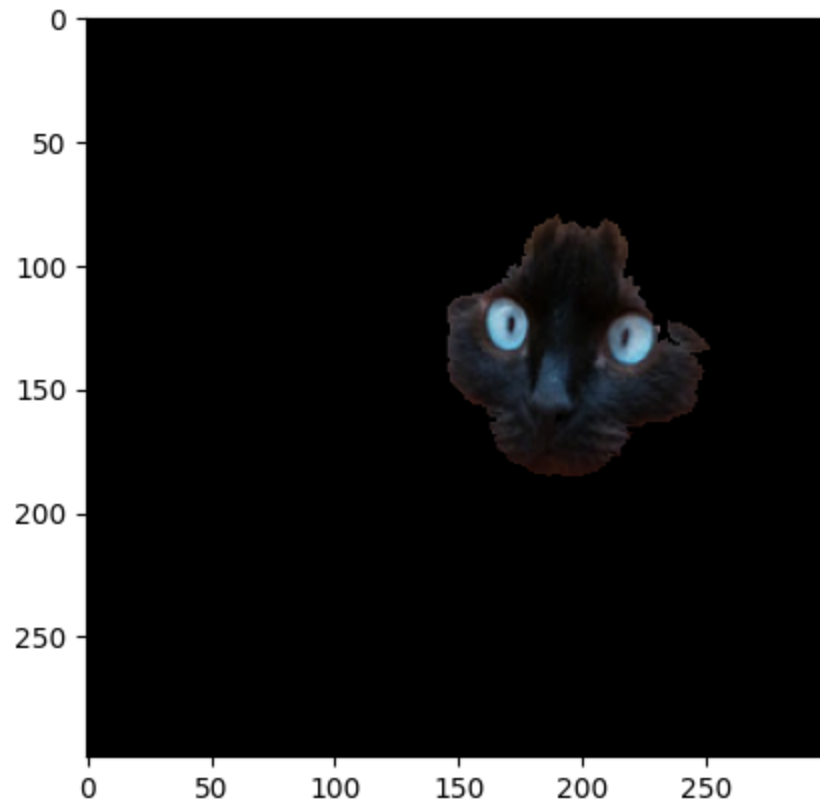
```
In [93]: i = 2 # you can change i to choose another picture
plt.imshow(data[i])
plt.show()
```



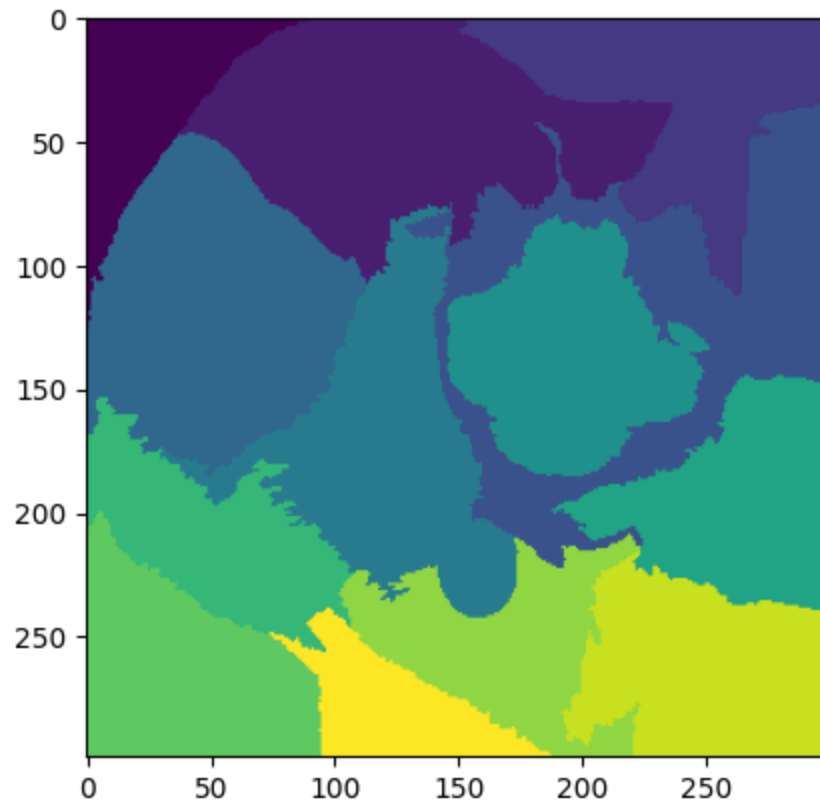
```
In [94]: image = images[i]
np.random.seed(0)
explanation = explainer.explain(image, threshold=.95, p_sample=.5, tau=0.25) # play around with parameters
```

```
1/1 [=====] - 0s 98ms/step
1/1 [=====] - 0s 98ms/step
4/4 [=====] - 2s 451ms/step
4/4 [=====] - 2s 469ms/step
```

```
In [95]: plt.imshow(explanation.anchor);
```



```
In [96]: plt.imshow(explanation.segments);
```



References

1. https://docs.seldon.io/projects/alibi/en/stable/examples/pdp_regression_bike.html
2. <https://docs.seldon.io/projects/alibi/en/stable/methods/PartialDependence.html>
3. https://docs.seldon.io/projects/alibi/en/stable/examples/anchor_image_imagenet.html